



WEEK 3 - PYTHON PROGRAMMING

Functions

Define reusable tasks, pass inputs and return results

Session 2 - 120 minutes, including a break

Instructor: MinGi KYUNG

Learning Goals & Agenda

- Define and call a function.
- Use parameters, arguments and return values.
- Distinguish printing a result from returning it.
- Explain local variables and compose small functions.

Concepts 12 min / Function basics 25 min / Arguments 16 min / Scope 7 min

Break 10 min / Functions with loops 20 min / Calculator lab 25 min / Review 5 min

Repeated Code, Different Inputs

- Both calculations follow the same rule.
- Only the input values change.
- A function gives this calculation a reusable name.
- The caller can supply different inputs each time.

```
price = 20
quantity = 3
print(price * quantity) # 60
```

```
price = 12
quantity = 5
print(price * quantity) # 60
```

Functions: A Name for a Task

A function is a named block of code that runs when called.

- Define: write the recipe once.
- Call: place an order to run it.
- Reuse: order again with new inputs.

Does writing a recipe make coffee?



Function Inputs and Results

- Arguments supply the input values.
- The function performs a task.
- `return` hands a result to the caller.

Example: `total_price(20, 3)` returns 60.

A function can take no arguments. Without an explicit return value, it returns `None`.

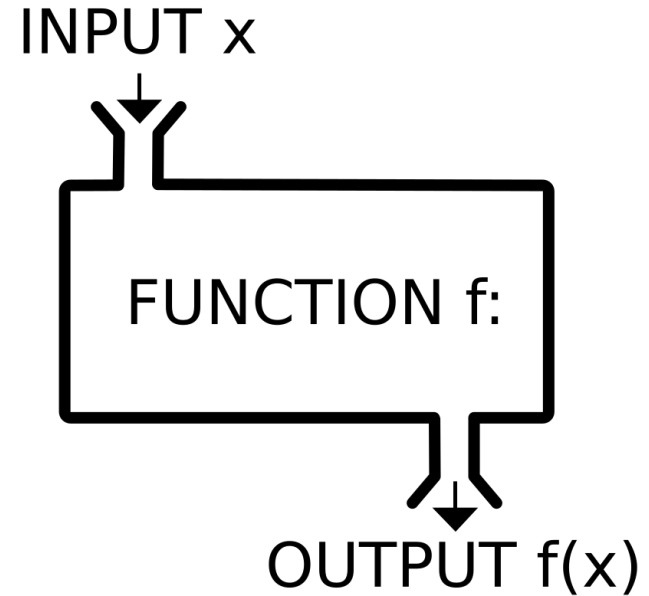


Image: Wvbailey / Wikimedia Commons / Public domain

Defining a Function

- `def` starts a function definition.
- Name the task, then put parameters in parentheses.
- A colon and indentation define the body.
- Defining the function does not run its body.

```
def greet():  
    print("Hello!")  
  
# No output until the function is called.  
greet()  
# Hello!
```

Calling a Function

- Parentheses make a function call.
- Execution enters the body and returns to the caller when the body finishes.
- Define a function before calling it in these scripts.
- Each call runs the body again.

```
def greet():  
    print("Hello!")  
  
print("Before")  
greet()  
greet()  
print("After")  
  
# Before  
# Hello!  
# Hello!  
# After
```

Parameters & Arguments

- Parameters are names in the function definition.
- Arguments are values supplied by the caller.
- Each call binds its arguments to the parameters.
- Parameters act as local variables.

```
def show_total(price, quantity):  
    print(price * quantity)  
  
show_total(20, 3)  # 60  
show_total(12, 5)  # 60  
  
# Parameters: price, quantity  
# Arguments in the first call: 20, 3
```

Returning a Result

- return sends a value back to the caller.
- Store the result in a variable or use it in an expression.
- return ends the current function call immediately.

```
def total_price(price, quantity):  
    return price * quantity  
  
amount = total_price(20, 3)  
print(amount)          # 60  
print(amount + 5)      # 65
```

Cafe Challenge: Where Is My Coffee?



```
def cafe():  
    print("Americano")  
  
drink = cafe()  
print(drink)
```

Vote before running the code.

Then replace print with return inside cafe().
What changes?

Can you drink an announcement?

What does drink hold: "Americano" or None?

print() and return

- print() displays information.
- return provides a value to the caller.
- A function can return a value without displaying anything.
- Choose return when another calculation needs the result.

```
def show_double(number):  
    print(number * 2)  
  
def double(number):  
    return number * 2  
  
shown = show_double(4)  # Displays 8  
saved = double(4)       # No output  
print(shown)            # None  
print(saved + 1)        # 9
```

Missing Return Values

- A function that reaches its end returns None.
- A bare return also returns None.
- Check that every intended result path returns a value.
- None is a special value, not the string "None".

```
def broken_double(number):  
    result = number * 2  
    # Missing: return result  
  
answer = broken_double(4)  
print(answer) # None  
  
# answer + 1 would raise TypeError.
```

Practice 1: An Even-Number Predicate

- Complete `is_even(number)` in `is_even.py`.
- Assume `number` is an integer.
- Return `True` for even numbers and `False` otherwise.
- Keep `input()` and `print()` outside the function.
- Test positive, zero and negative inputs.

```
def is_even(number):  
    # Return a Boolean result.  
    pass  
  
print(is_even(4))    # Expected: True  
print(is_even(7))    # Expected: False  
print(is_even(0))    # Expected: True  
print(is_even(-2))   # Expected: True
```

Positional & Keyword Arguments

- Positional arguments bind in parameter order.
- Keyword arguments name the parameter explicitly.
- Keyword arguments can clarify a call.
- For these signatures, put positional arguments before keyword arguments.

```
def total_price(price, quantity):  
    return price * quantity  
  
print(total_price(20, 3))  
print(total_price(price=20, quantity=3))  
print(total_price(quantity=3, price=20))  
print(total_price(20, quantity=3))  
# Every call prints 60.
```

Default Parameter Values

- A default supplies a value when the caller omits an argument.
- The caller can override it explicitly.
- Put required parameters before defaulted ones in these signatures.
- Use simple numeric defaults in today's examples.

```
def power(base, exponent=2):  
    return base ** exponent  
  
print(power(3))           # 9  
print(power(2, 3))        # 8  
print(power(5, exponent=0)) # 1
```

Practice 2: A Power Function

- Complete `power(base, exponent=2)` in `power.py`.
- Return base raised to exponent.
- Assume base is a number and exponent is a nonnegative integer.
- Verify the default and both styles of explicit argument.

Call	Expected
<code>power(3)</code>	9
<code>power(2, 3)</code>	8
<code>power(5, exponent=0)</code>	1
<code>power(-2, 3)</code>	-8

Use `**` for exponentiation.
Do not print inside the function.

Local Variables & Scope

- A name assigned inside a function is local in these examples.
- A local name can differ from a same-named variable outside.
- Return values to transfer results explicitly.
- Avoid global state in today's functions.

```
number = 10

def add_one(number):
    result = number + 1
    return result

print(add_one(3)) # 4
print(number)     # 10

# print(result) would raise NameError.
```

Break & Midpoint Check

10-minute break

- Does defining a function run its body?
- What is the difference between a parameter and an argument?
- What does a function return if it has no return statement?
- Can a caller use a local variable directly after the call ends?

Next: wrapping a loop in a function.

A Loop Inside a Function

- A function body can contain a loop.
- Initialize the accumulator on every call.
- Place return after the loop to return the complete sum.
- This version assumes n is a nonnegative integer.

```
def sum_to(n):  
    total = 0  
    for value in range(1, n + 1):  
        total += value  
    return total  
  
print(sum_to(5)) # 15  
print(sum_to(3)) # 6  
print(sum_to(0)) # 0
```

The Barista Who Leaves Early

- Three customers are waiting for coffee.
- Predict: how many messages will this code display?
- Vote: zero, one or three. Then trace the first iteration.
- A return ends the current function call immediately.

```
def serve():  
    for number in range(1, 4):  
        return f"Coffee for {number}"  
  
print(serve())
```

What changes if return becomes print inside the loop?
Also change the last line to serve().

return Ends the Whole Function

- return exits the function, including any loop inside it.
- break exits only the nearest loop.
- Returning inside the loop can stop accumulation too early.

```
def too_early(n):  
    total = 0  
    for value in range(1, n + 1):  
        total += value  
        return total # Wrong indentation  
  
print(too_early(5)) # 1, not 15  
print(too_early(0)) # None
```

A Function's Input and Output Rules

- Define the allowed inputs and expected output before coding.
- For `sum_to`, accept nonnegative integers.
- For a negative integer, return `None` to signal invalid input.
- Assume callers supply integers; type-error handling comes later.

```
def sum_to(n):  
    if n < 0:  
        return None  
    total = 0  
    for value in range(1, n + 1):  
        total += value  
    return total  
  
print(sum_to(-1)) # None  
print(sum_to(5))  # 15
```

Lab 3: Sum from 1 to n

- Complete `sum_to(n)` in `sum_to.py` using a loop.
- Return the sum of integers from 1 through `n`.
- Return 0 for `n = 0` and `None` for negative `n`.
- Do not use `sum()` or the arithmetic-series formula.
- Keep printing in the caller.

Use `is None` to check the special `None` value.

```
def sum_to(n):  
    # Validate, accumulate, then return.  
    pass  
  
result = sum_to(5)  
if result is None:  
    print("Invalid input")  
else:  
    print("Total:", result)
```

Lab 3 Checks

- Test the boundary $n = 0$ and a negative input.
- Call the function more than once.
- Each call must start with a fresh total.
- Do not confuse a valid result of 0 with None.

Call	Expected return value
<code>sum_to(1)</code>	1
<code>sum_to(5)</code>	15
<code>sum_to(0)</code>	0
<code>sum_to(-1)</code>	None
<code>sum_to(3)</code>	6
<code>sum_to(5)</code>	15 again

Combining Small Functions

- A function's return value can become another function's argument.
- Keep each function's purpose clear.
- This example calculates a price, then adds a fee.

```
def total_price(price, quantity):  
    return price * quantity  
  
def add_fee(amount, fee=5):  
    return amount + fee  
  
subtotal = total_price(20, 3)  
final = add_fee(subtotal)  
print(final)  # 65
```

Calculator Function Requirements

- Define `calculate(a, b, operator)`.
- Support "+", "-", "*" and "/".
- Assume a and b are numbers and operator is a string.
- Return `None` for an unsupported operator or division by zero.
- Return a numeric result for valid operations, including zero.

Call	Returns
<code>calculate(8, 2, "+")</code>	10
<code>calculate(8, 2, "-")</code>	6
<code>calculate(8, 2, "*")</code>	16
<code>calculate(8, 2, "/")</code>	4.0
<code>calculate(8, 0, "/")</code>	<code>None</code>
<code>calculate(8, 2, "%")</code>	<code>None</code>

Lab 4: Calculator Function

- Complete calculator.py using if, elif and return.
- Keep input and output outside calculate().
- Check for None before displaying a numeric result.
- A valid result of zero must display normally.

Optional: read a, b and operator in the caller, then call calculate().

```
def calculate(a, b, operator):  
    # Return a result or None.  
    pass  
  
result = calculate(8, 2, "+")  
if result is None:  
    print("Invalid operation")  
else:  
    print("Result:", result)
```

Lab 4 Checks

- Check all four supported operations.
- Test division by zero and an unsupported operator.
- Test negative values and a result of zero.
- Functions should return values without printing.

Additional cases

```
calculate(2, 2, "-")    0
calculate(-3, 2, "*")   -6
calculate(0, 5, "/")    0.0
```

Avoid: `if not result`
It treats a valid zero as false.
Use: `if result is None`

Common Function Mistakes

- Writing a function name without parentheses does not call it.
- Passing too few required arguments raises `TypeError`.
- Printing a value does not return it to the caller.
- A return inside a loop may finish the task too early.
- A local variable is not directly available outside its function.

For each bug, identify the definition, the call and the returned value.

Testing & Submission

Submit `is_even.py`, `power.py`, `sum_to.py` and `calculator.py`.

- Keep the specified function names and parameter names.
- Return values from functions and display them in the caller.
- Add comments recording inputs and expected return values.
- Include boundary cases and the invalid cases required by the contract.
- Confirm repeated calls produce independent results.

Collections and exception handling are not required for these exercises.

Exit Quiz

1. In `def power(base, exponent=2)`, which names are parameters?
2. What does `power(3)` return?
3. What does a function without return produce?
4. Why is return after the loop important in `sum_to()`?
5. Why use `result is None` instead of `not result` in the calculator?

Explain how the caller receives a value from a function.

Next Session: Modules

- A module groups reusable code in a Python file.
- Use import to access a module's functions.
- Explore standard library modules such as math and random.
- Put your own functions in a module and reuse them in another file.

```
import math  
  
print(math.sqrt(25))  # 5.0
```